COMPLETE STUDY & REVISION GUIDE

Decision and Case Control Structures

if • if-else • nested if • else-if construct • switch-case • ternary operator

C

Logical & Relational Operators | Compound Relational Tests | Flowcharts | Truth Tables | Solved Programs with Output | Comparison Tables | Quick Revision Notes

For Competitive Examinations | University Examinations | Teacher Recruitment Tests

Table of Contents

1. Introduction to Decision Control Structures 3
2. The `if` Statement 4
3. The `if-else` Statement 6
4. Logical Operators & Compound Relational Tests 8
5. Nested `if` Statements 11
6. The `else-if` Construct 13
7. Relational Operators 15
8. Conditional (Ternary) Operator 16
9. The `switch` Statement 18
10. Comparison Section 22
11. Solved Programming Examples 24
12. Quick Revision Notes (One-Page Summary) 27

How to use this guide

Each chapter follows the same structure — Definition → Syntax → Working Mechanism → Flowchart → Solved Examples → Advantages/Limitations → Common Errors — so that it can be used both for first-time learning and for last-minute exam revision.

CHAPTER 1

Introduction to Decision Control Structures

1.1 Definition and Importance

Decision control structures are programming constructs that allow a program to choose between two or more paths of execution based on whether a given condition evaluates to **true** or **false**. They form the basis of "intelligent" behaviour in a program, allowing it to react differently to different inputs.

> **Why are they important?**
> Without decision control, a program would execute every statement in a fixed, linear sequence regardless of input. Decision structures let a program validate data, branch logic, select one of many actions, and implement real-world rules (eligibility, grading, menus, etc.).

1.2 Flow of Decision-Making in a Program

Every decision statement in C works on the same underlying principle:

1. A **condition** (a relational/logical expression) is evaluated.
2. In C, any non-zero value is treated as **true** and zero is treated as **false**.
3. Depending on the result, control is transferred to one of the available branches.
4. Execution continues from the statement following the decision block.

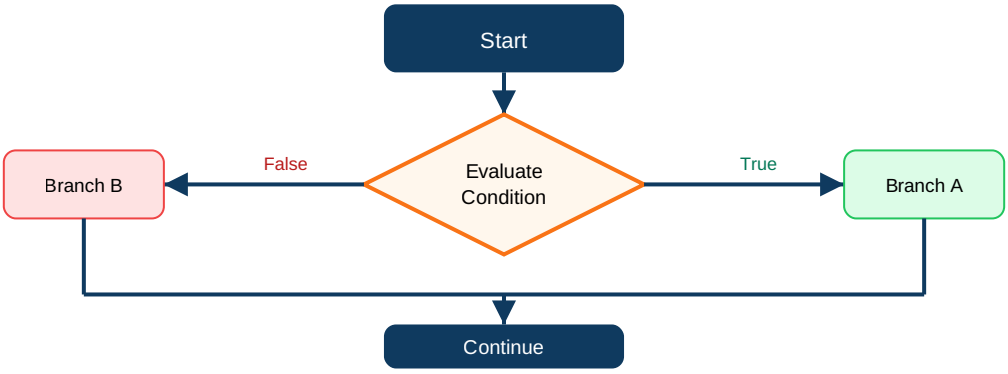


Fig 1.1 — General flow of decision-making in a program

1.3 Types of Decision Control Statements in C

Category	Statements	Use Case
Simple Decision	`if`, `if-else`	Two-way branching
Multi-way Decision	Nested `if`, `else-if` construct, `switch`	Choosing among many alternatives
Expression-based Decision	Conditional (ternary) operator `?:`	Compact single-expression decisions

Chapter Summary

- Decision control structures let a program choose between alternative actions based on conditions.
- In C, zero = false and any non-zero value = true.
- C provides three broad families: simple (`if`/`if-else`), multi-way (nested `if`, `else-if`, `switch`), and expression-based (ternary).

CHAPTER 2

The `if` Statement

2.1 Definition

The `if` statement is the simplest decision control structure in C. It executes a block of statements **only if** the given condition evaluates to true; otherwise, the block is skipped entirely.

2.2 Syntax

SYNTAX

```
if (condition)
{
    statement(s);   // executed only when condition is true
}
// control resumes here regardless of condition
```

Note

If only a single statement follows `if`, the braces `{ }` are optional — but using them is recommended exam-safe practice to avoid the classic "dangling else" / wrong-statement bugs.

Short Note: Are { } Mandatory in if-else?

No — braces are optional for a **single statement**, mandatory for **two or more statements**.

```
if (n % 2 == 0) printf("Even"); else printf("Odd");  // valid
```

Rule: Not required for single statements — but **always recommended** to avoid bugs.

2.3 Working Mechanism

1. The expression inside `( )` is evaluated.
2. If the result is non-zero (true), the statement block executes.
3. If the result is zero (false), the block is skipped.
4. Execution continues with the statement immediately after the `if` block.

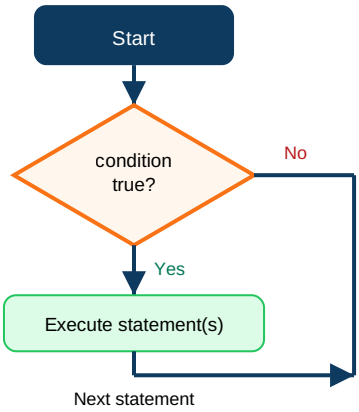


Fig 2.1 — Flowchart of the `if` statement

2.4 Example with Output

PROGRAM: CHECK IF A NUMBER IS POSITIVE

```
#include <stdio.h>
int main()
{
    int num = 15;
    if (num > 0)
    {
        printf("%d is a positive number", num);
    }
    return 0;
}
```

OUTPUT

```
15 is a positive number
```

2.5 Advantages and Limitations

Advantages	Limitations
Simple and easy to understand for one-way checks	Cannot specify an alternate action when the condition is false
Useful for validation/guard checks	Becomes unwieldy for multiple conditions (needs nesting or ladder)

Common Error

Placing a semicolon right after `if(condition);` creates an empty statement — the block below executes unconditionally, which is a very common exam trick question.

Chapter Summary

- `if` provides one-way decision making — executes a block only when its condition is true.
- Braces are optional for a single statement but recommended.
- A stray semicolon after `if(condition)` is a classic bug.

CHAPTER 3

The `if-else` Statement

3.1 Definition

The `if-else` statement provides **two-way branching**: one block executes when the condition is true, and a different block executes when it is false. Exactly one of the two blocks always runs.

3.2 Syntax

```
SYNTAX
if (condition)
{
    statement(s);   // executed when condition is true
}
else
{
    statement(s);   // executed when condition is false
}
```

3.3 Execution Process

1. Condition is evaluated.
2. If true → the `if` block executes and the `else` block is skipped.
3. If false → the `else` block executes and the `if` block is skipped.
4. Control resumes after the entire `if-else` construct.

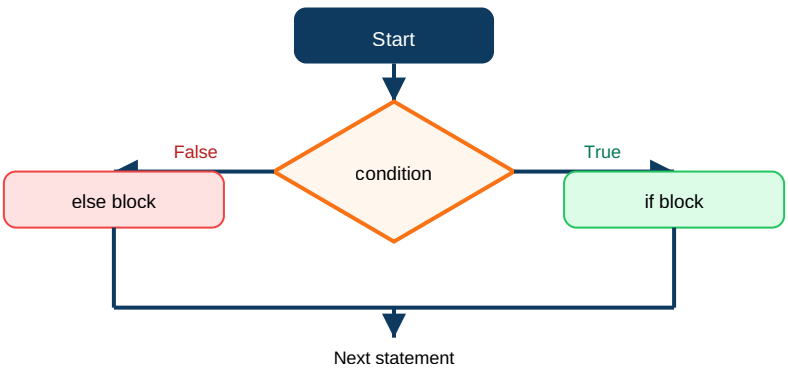


Fig 3.1 — Flowchart of the `if-else` statement

3.4 Program with Output

PROGRAM: CHECK EVEN OR ODD

```
#include <stdio.h>
int main()
{
    int n = 7;
    if (n % 2 == 0)
        printf("%d is Even", n);
    else
        printf("%d is Odd", n);
    return 0;
}
```

OUTPUT

```
7 is Odd
```

EXAM TIP

In an `if-else` chain, the `else` always binds to the **nearest unmatched** preceding `if` — this is the basis of "dangling else" questions.

Chapter Summary

- `if-else` gives two-way branching: exactly one branch always executes.
- `else` never has its own condition — it simply catches the "otherwise" case.
- Watch for the dangling-else rule in nested constructs.

CHAPTER 4

Logical Operators & Compound Relational Tests

4.1 Why Logical Operators?

Most real-world decisions require **two or more conditions** to be evaluated together (e.g., "age is above 18 AND has a valid ID"). C provides three logical operators to combine relational expressions into a single compound condition.

Operator	Name	Meaning	Example
&&	Logical AND	True only if **both** operands are true	`(a>0 && b>0)`
||	Logical OR	True if **at least one** operand is true	`(a>0 || b>0)`
!	Logical NOT	Reverses (negates) the truth value	`!(a>0)`

4.2 Truth Tables

A	B	A && B
True	True	True
True	False	False
False	True	False
False	False	False

A	B	A || B
True	True	True
True	False	True
False	True	True
False	False	False

A	!A
True	False
False	True

4.3 Compound Relational Tests

A **compound relational test** combines two or more relational expressions (using <, >, ==, etc.) with logical operators to form a single condition.

```
SYNTAX
if (variable1 < value1 && variable2 > value2)
{
    statement(s);
}
```

Common Error

Writing `if (10 < x < 20)` does NOT check a range in C — it evaluates left to right: `(10<x)` gives 0 or 1, which is then compared with 20 (always true). The correct compound form is `if (x > 10 && x < 20)`.

4.4 Short-Circuit Evaluation

C evaluates && and || using **short-circuit evaluation**: for &&, if the first operand is false, the second is never evaluated (result is already false); for ||, if the first operand is true, the second is skipped.

4.5 Practical Example

PROGRAM: ELIGIBILITY CHECK USING COMPOUND CONDITION

```
#include <stdio.h>
int main()
{
    int age = 20;
    char hasID = 'Y';

    if (age >= 18 && hasID == 'Y')
        printf("Eligible to vote");
    else
        printf("Not eligible to vote");
    return 0;
}
```

OUTPUT

```
Eligible to vote
```

PROGRAM: LEAP YEAR USING COMPOUND RELATIONAL TEST

```
#include <stdio.h>
int main()
{
    int year = 2024;
    if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))
        printf("%d is a Leap Year", year);
    else
        printf("%d is not a Leap Year", year);
    return 0;
}
```

OUTPUT

```
2024 is a Leap Year
```

Chapter Summary

- && needs all conditions true; || needs at least one true; ! reverses a condition.
- Compound relational tests combine multiple relational expressions using logical operators.
- C uses short-circuit evaluation — second operand may not be evaluated.
- `10 < x < 20` is NOT a valid range check in C.

CHAPTER 5

Nested `if` Statements

5.1 Concept

A **nested if** is an `if` or `if-else` statement placed entirely inside the body of another `if` or `else` block. It is used when a decision must be made only after another condition has already been satisfied — i.e., conditions that depend on each other rather than being independent.

5.2 Syntax

SYNTAX

```
if (condition1)
{
    if (condition2)
    {
        statement(s);   // both condition1 and condition2 true
    }
    else
    {
        statement(s);   // condition1 true, condition2 false
    }
}
else
{
    statement(s);       // condition1 false
}
```

5.3 Working Process

1. Outer condition is evaluated first.
2. The inner condition is evaluated only if the outer condition is true.
3. If the outer condition is false, the inner `if` is never tested at all.

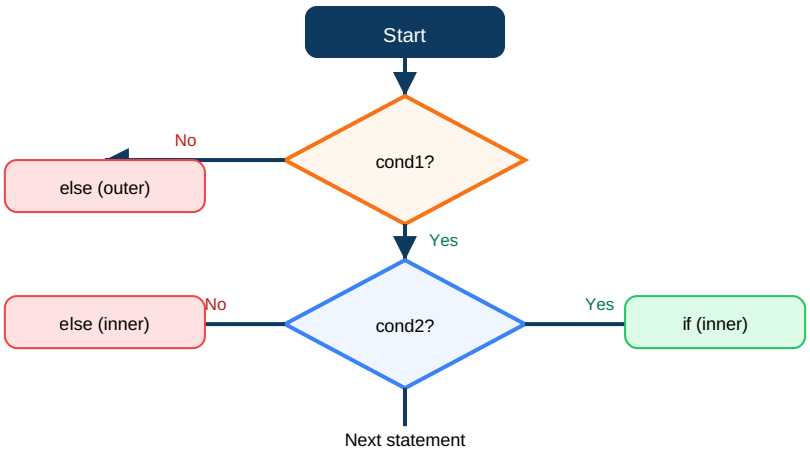


Fig 5.1 — Flowchart of a nested `if` statement

5.4 Sample Program

PROGRAM: LARGEST OF TWO NUMBERS USING NESTED IF

```
#include <stdio.h>
int main()
{
    int a = 25, b = 40;
    if (a != b)
    {
        if (a > b)
            printf("%d is largest", a);
        else
            printf("%d is largest", b);
    }
    else
        printf("Both numbers are equal");
    return 0;
}
```

OUTPUT

```
40 is largest
```

EXAM TIP

Nested `if` and compound conditions (&&) can often achieve the same result, but nesting allows different actions per individual sub-condition, while && only tells you whether all conditions together are true.

Chapter Summary

- Nested `if` places one `if` inside another to test dependent conditions.
- The inner condition is evaluated only when the outer condition is true.
- Always indent nested blocks clearly to avoid dangling-else confusion.

CHAPTER 6

The `else-if` Construct

6.1 Definition

The `else-if` construct (also called the **if-else-if** staircase) tests a series of conditions **one after another** in sequence and executes the block for the **first** condition found true. If none match, the final optional `else` executes.

6.2 Syntax

SYNTAX

```
if (condition1)
    statement1;
else if (condition2)
    statement2;
else if (condition3)
    statement3;
...
else
    default_statement;   // executes if none of the above are true
```

6.3 Execution Sequence

1. Conditions are tested strictly top-to-bottom.
2. As soon as one condition is true, its block runs and **all remaining conditions are skipped**.
3. If no condition is true, the final `else` block (if present) runs.

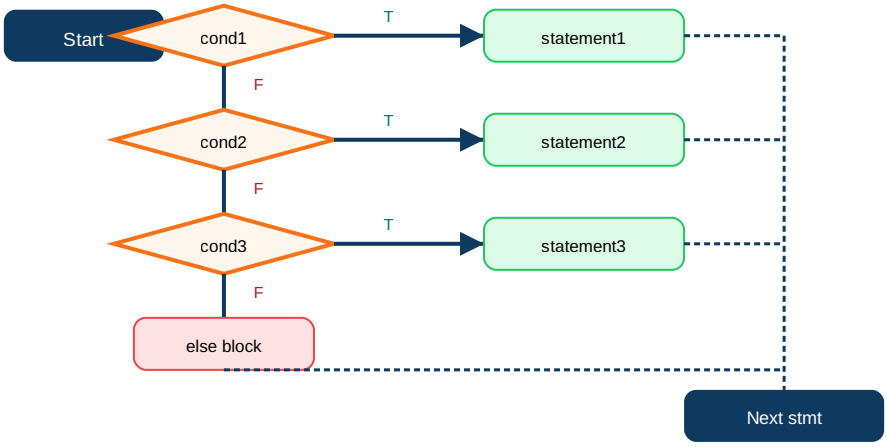


Fig 6.1 — Flowchart of the `else-if` construct

6.4 Example with Output

PROGRAM: GRADE CALCULATION USING ELSE-IF CONSTRUCT

```
#include <stdio.h>
int main()
{
    int marks = 78;
    if (marks >= 90)
        printf("Grade: A");
    else if (marks >= 75)
        printf("Grade: B");
    else if (marks >= 60)
        printf("Grade: C");
    else
        printf("Grade: F");
    return 0;
}
```

OUTPUT

```
Grade: B
```

Common Error

Writing overlapping or wrongly ordered conditions (e.g., checking `marks >= 60` before `marks >= 90`) causes higher grades to never be reached, since the ladder stops at the first true condition.

Chapter Summary

- The `else-if` construct tests multiple conditions in sequence and runs only the first matching block.
- Order of conditions matters — place the most specific/restrictive condition first.
- The final `else` acts as a default/catch-all case.

CHAPTER 7

Relational Operators

7.1 Definition

Relational operators compare two values/expressions and yield a result of `1` (true) or `0` (false). They form the building blocks of every condition used in `if`, `else-if`, and loop statements.

7.2 List of Relational Operators

Operator	Meaning	Example	Result (a=10, b=20)
<	Less than	`a < b`	1 (true)
>	Greater than	`a > b`	0 (false)
<=	Less than or equal to	`a <= 10`	1 (true)
>=	Greater than or equal to	`a >= b`	0 (false)
==	Equal to	`a == b`	0 (false)
!=	Not equal to	`a != b`	1 (true)

Common Error

Confusing the assignment operator = with the equality operator ==. Writing `if (a = 5)` assigns 5 to a and the condition is always true (since 5 is non-zero) — a notorious source of bugs and exam questions.

7.3 Example

PROGRAM: DEMONSTRATING RELATIONAL OPERATORS

```
#include <stdio.h>
int main()
{
    int a = 10, b = 20;
    printf("%d\n", a < b);    // 1
    printf("%d\n", a > b);    // 0
    printf("%d\n", a == b);   // 0
    printf("%d\n", a != b);   // 1
    return 0;
}
```

OUTPUT

```
1
0
0
1
```

Chapter Summary

- Relational operators compare two operands and return 1 (true) or 0 (false).
- The six relational operators are `<` `>` `<=` `>=` `==` `!=`.
- Never confuse `=` (assignment) with `==` (comparison).

CHAPTER 8

Conditional (Ternary) Operator

8.1 Definition

The **conditional operator** `?:` is C's only ternary (three-operand) operator. It evaluates a condition and returns one of two expressions/values depending on whether the condition is true or false — all in a single expression.

8.2 Syntax

SYNTAX

```
condition ? expression_if_true : expression_if_false;
```

8.3 Working Mechanism

1. `condition` is evaluated first.
2. If true, the entire expression evaluates to `expression_if_true`.
3. If false, the entire expression evaluates to `expression_if_false`.
4. The result can be assigned to a variable, can be printed, or can be used in another expression.

8.4 Advantages

- Compact — replaces a 4-line `if-else` with one line.
- Can be used directly inside expressions, function arguments, and `printf()` calls (unlike `if-else`, which is a statement, not an expression).
- Ternary expressions can be nested for multiple conditions, though readability suffers.

8.5 Comparison with `if-else`

Aspect	`if-else`	Ternary Operator `?:`
Type	Statement	Expression (returns a value)
Usage	Can hold multiple statements per branch	Limited to single expressions per branch
Readability	Better for complex logic	Best for simple, single-value decisions
Nesting	Easy to nest and read	Becomes unreadable when deeply nested
Can be used inside `printf()`?	No	Yes

8.6 Example Programs

PROGRAM: LARGEST OF TWO NUMBERS USING TERNARY OPERATOR

```
#include <stdio.h>
int main()
{
    int a = 35, b = 22, large;
    large = (a > b) ? a : b;
    printf("Largest number is %d", large);
    return 0;
}
```

OUTPUT

```
Largest number is 35
```

PROGRAM: EVEN/ODD CHECK USING TERNARY OPERATOR DIRECTLY IN PRINTF

```
#include <stdio.h>
int main()
{
    int n = 9;
    printf("%d is %s", n, (n % 2 == 0) ? "Even" : "Odd");
    return 0;
}
```

OUTPUT

```
9 is Odd
```

EXAM TIP

The conditional operator is right-associative, which allows chaining: `a > b ? a : (b > c ? b : c)` finds the largest of three numbers in one line.

Chapter Summary

- `?:` is C's only ternary operator: `condition ? true_expr : false_expr`.
- It is an expression (yields a value), unlike `if-else` which is a statement.
- Best suited for simple, single-value two-way decisions.

CHAPTER 9

The `switch` Statement

9.1 Definition

The `switch` statement is a **multi-way branching** construct that compares a single expression against a list of constant integer/character values (**case labels**) and transfers control to the matching case.

9.2 Syntax

```
SYNTAX
switch (expression)
{
    case constant1:
        statement(s);
        break;
    case constant2:
        statement(s);
        break;
    ...
    default:
        statement(s);
}
```

9.3 Concept of a Case Label

A **case label** (`case constant:`) is a marker that identifies a specific entry point inside the `switch` block. The value after `case` must be a **constant expression** (integer or character literal) — variables, floats, and ranges are not allowed as case labels in standard C.

Rules for Case Labels

- Must be a constant integer or character expression — no variables, no floats.
- No two case labels in the same switch may have the same value.
- The `default` label is optional and can appear anywhere, but conventionally goes last.

9.4 Working Process

1. The `switch` expression is evaluated once.
2. Control jumps directly to the `case` label whose constant matches the expression's value.
3. Statements execute from that label onward until a `break` is met or the switch block ends.
4. If no case matches, the `default` block (if present) executes.

9.5 The `break` Statement

The `break` statement immediately exits the `switch` block, preventing execution from continuing into the next case. Without it, execution "falls through" to subsequent cases.

9.6 Switch With `break` — Example

PROGRAM: DAY NAME USING SWITCH WITH BREAK

```
#include <stdio.h>
int main()
{
    int day = 3;
    switch (day)
    {
        case 1: printf("Monday");    break;
        case 2: printf("Tuesday");   break;
        case 3: printf("Wednesday"); break;
        case 4: printf("Thursday");  break;
        default: printf("Invalid day");
    }
    return 0;
}
```

OUTPUT

```
Wednesday
```

9.7 Switch Without `break` — Fall-Through Concept

If `break` is omitted, control "falls through" and keeps executing every subsequent case's statements regardless of their labels, until a `break` or the end of the switch is reached. This behaviour is sometimes used deliberately to group multiple cases together.

PROGRAM: FALL-THROUGH BEHAVIOUR (NO BREAK)

```
#include <stdio.h>
int main()
{
    int n = 2;
    switch (n)
    {
        case 1: printf("One\n");
        case 2: printf("Two\n");
        case 3: printf("Three\n");
        default: printf("Default\n");
    }
    return 0;
}
```

OUTPUT

```
Two
Three
Default
```

Why this output?

Execution starts at `case 2` (the match) and then "falls through" every line below it since no `break` stops it — printing Two, Three, and Default in sequence.

9.8 Deliberate Fall-Through — Grouping Cases

PROGRAM: VOWEL CHECK USING GROUPED CASE LABELS

```
#include <stdio.h>
int main()
{
    char ch = 'e';
    switch (ch)
    {
        case 'a':
        case 'e':
        case 'i':
        case 'o':
        case 'u':
            printf("%c is a Vowel", ch);
            break;
        default:
            printf("%c is a Consonant", ch);
    }
    return 0;
}
```

OUTPUT

```
e is a Vowel
```

9.9 Flowchart

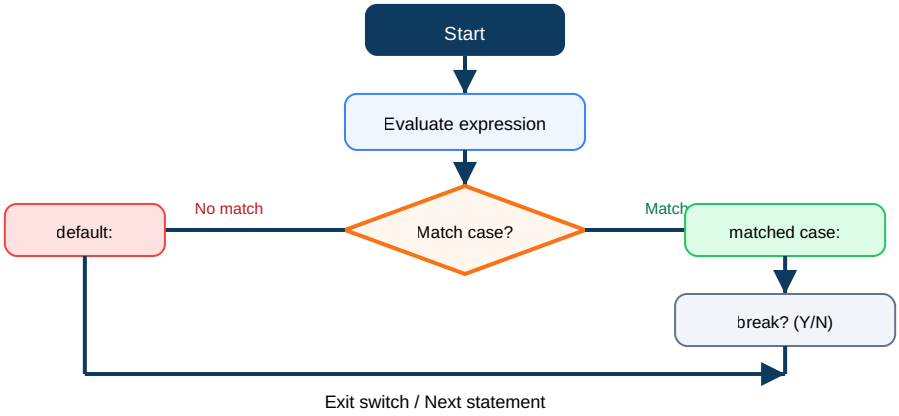


Fig 9.1 — Flowchart of the `switch` statement

9.10 Switch vs. If-Else Construct — Quick Notes

- `switch` can only test **equality** against constants; it cannot evaluate ranges or compound logical conditions directly.

- `switch` works only with `int` or `char` (integral) expressions — not `float`/`double`/strings in standard C.
- `switch` is often faster and more readable than a long `else-if` construct for many discrete, equality-based choices (e.g., menu-driven programs).

Common Errors

- Forgetting `break` when fall-through is NOT intended.
- Using a non-constant (variable) value as a case label — causes a compile-time error.
- Duplicate case values in the same switch — compile-time error.

Chapter Summary

- `switch` performs multi-way branching by matching an expression against constant case labels.
- `break` exits the switch; omitting it causes fall-through to subsequent cases.
- `default` handles unmatched values and is optional.
- Case labels must be unique constant integer/character values.

CHAPTER 10

Comparison Section

10.1 `if` vs `if-else`

Feature	`if`	`if-else`
Branches	One (true branch only)	Two (true and false)
Alternate action when condition is false	None — skips block	Yes — executes `else` block
Use case	Guard checks, one-way actions	Binary decisions
Always one branch runs?	No — may do nothing	Yes — exactly one branch always runs

10.2 `else-if` Ladder vs `switch`

Feature	`else-if` Ladder	`switch`
Type of test	Any relational/logical expression (ranges, compound)	Equality against constant integral values only
Data types	int, float, char, expressions	int, char only (no float/string)
Readability for many cases	Can get long and cluttered	Cleaner and easier to read
Fall-through behaviour	Not applicable	Falls through if `break` is omitted
Default/catch-all	Final `else`	`default:` label
Performance (large number of cases)	Linear scan (slower)	Jump table (potentially faster)
Best use	Grade ranges, range checks, complex logic	Menu-driven programs, day/month/character selection

10.3 `if-else` vs Ternary Operator

Feature	`if-else`	Ternary `?:`
Category	Statement	Expression (returns a value)
Lines of code	4+ lines	1 line
Usable inside expressions / printf()	No	Yes
Multiple statements per branch	Yes	No — one expression per branch
Readability for complex logic	Better	Gets confusing when nested
Best use	Complex, multi-statement decisions	Simple single-value decisions

10.4 Summary of All Decision Constructs

Construct	Branches	Condition Type	Best Used When
`if`	1	Any boolean expression	One-way action, guard checks
`if-else`	2	Any boolean expression	Binary (yes/no) decisions
Nested `if`	Multiple (hierarchical)	Any boolean expression	Dependent/sequential conditions
`else-if` construct	Many (sequential)	Any boolean expression	Range checks, ordered priorities
Ternary `?:`	2	Any boolean expression	Compact single-value decision
`switch`	Many (direct jump)	Integer/char equality only	Menu, categories, discrete values

CHAPTER 11

Solved Programming Examples

11.1 Even / Odd Checking

METHOD 1: USING IF-ELSE

```
#include <stdio.h>
int main()
{
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    if (n % 2 == 0)
        printf("%d is Even", n);
    else
        printf("%d is Odd", n);
    return 0;
}
```

OUTPUT (INPUT: 14)

```
14 is Even
```

METHOD 2: USING TERNARY OPERATOR

```
#include <stdio.h>
int main()
{
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    printf("%d is %s", n, (n % 2 == 0) ? "Even" : "Odd");
    return 0;
}
```

11.2 Largest of Three Numbers

PROGRAM: LARGEST OF 3 USING NESTED IF-ELSE

```
#include <stdio.h>
int main()
{
    int a = 45, b = 72, c = 36;
    if (a > b)
    {
        if (a > c)
            printf("%d is the largest", a);
        else
            printf("%d is the largest", c);
    }
    else
    {
        if (b > c)
            printf("%d is the largest", b);
        else
            printf("%d is the largest", c);
    }
    return 0;
}
```

OUTPUT

```
72 is the largest
```

11.3 Grade Calculation (else-if Construct)

PROGRAM: PERCENTAGE-BASED GRADE USING ELSE-IF CONSTRUCT

```
#include <stdio.h>
int main()
{
    float percent;
    printf("Enter percentage: ");
    scanf("%f", &percent);

    if (percent >= 90)
        printf("Grade: A (Distinction)");
    else if (percent >= 75)
        printf("Grade: B (First Class)");
    else if (percent >= 60)
        printf("Grade: C (Second Class)");
    else if (percent >= 45)
        printf("Grade: D (Pass)");
    else
        printf("Grade: F (Fail)");
    return 0;
}
```

OUTPUT (INPUT: 81.5)

```
Grade: B (First Class)
```

11.4 Menu-Driven Program Using switch

PROGRAM: CALCULATOR USING SWITCH-CASE

```
#include <stdio.h>
int main()
{
    int a = 20, b = 4, choice;
    printf("1.Add  2.Subtract  3.Multiply  4.Divide\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1: printf("Result = %d", a + b); break;
        case 2: printf("Result = %d", a - b); break;
        case 3: printf("Result = %d", a * b); break;
        case 4:
            if (b != 0)
                printf("Result = %d", a / b);
            else
                printf("Division by zero!");
            break;
        default: printf("Invalid choice!");
    }
    return 0;
}
```

OUTPUT (CHOICE: 3)

```
Result = 80
```

11.5 Leap Year Check (Compound Relational Test)

PROGRAM: LEAP YEAR USING LOGICAL OPERATORS

```
#include <stdio.h>
int main()
{
    int year;
    printf("Enter year: ");
    scanf("%d", &year);

    if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))
        printf("%d is a Leap Year", year);
    else
        printf("%d is NOT a Leap Year", year);
    return 0;
}
```

OUTPUT (INPUT: 2000)

```
2000 is a Leap Year
```

11.6 Maximum of Two Numbers — Ternary Operator

PROGRAM: LARGEST OF 3 USING CHAINED TERNARY

```
#include <stdio.h>
int main()
{
    int a = 18, b = 45, c = 30, max;
    max = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);
    printf("Largest = %d", max);
    return 0;
}
```

OUTPUT

```
Largest = 45
```

11.7 Character Category Check Using switch

PROGRAM: CHECK VOWEL / CONSONANT / DIGIT USING SWITCH

```
#include <stdio.h>
int main()
{
    char ch = 'A';
    switch (ch)
    {
        case 'a': case 'e': case 'i': case 'o': case 'u':
        case 'A': case 'E': case 'I': case 'O': case 'U':
            printf("%c is a Vowel", ch);
            break;
        case '0': case '1': case '2': case '3': case '4':
        case '5': case '6': case '7': case '8': case '9':
            printf("%c is a Digit", ch);
            break;
        default:
            printf("%c is a Consonant or Special Character", ch);
    }
    return 0;
}
```

OUTPUT

```
A is a Vowel
```

11.8 Positive / Negative / Zero

PROGRAM: SIGN CHECK USING ELSE-IF CONSTRUCT

```
#include <stdio.h>
int main()
{
    int n = -7;
    if (n > 0)
        printf("%d is Positive", n);
    else if (n < 0)
        printf("%d is Negative", n);
    else
        printf("Number is Zero");
    return 0;
}
```

OUTPUT

```
-7 is Negative
```

CHAPTER 12

Quick Revision Notes

12.1 Important Syntax Summary

Construct	Minimum Syntax
`if`	`if(cond) stmt;`
`if-else`	`if(cond) stmt1; else stmt2;`
Nested `if`	`if(c1){ if(c2) stmt; }`
`else-if` construct	`if(c1) s1; else if(c2) s2; else s3;`
Ternary operator	`variable = (cond) ? val_true : val_false;`
`switch`	`switch(expr){ case C: stmt; break; default: stmt; }`

12.2 Key Concepts at a Glance

- In C, **zero = false** and **any non-zero = true**.
- Relational operators return **1** (true) or **0** (false).
- == is comparison; = is assignment — confusing them is a classic C bug.
- The `else` binds to the nearest unmatched `if` (dangling-else rule).
- && and || use **short-circuit evaluation**.
- Range check: always use `x > 10 && x < 20`, never `10 < x < 20`.
- `switch` works only on **integer/character** types — not float/string.
- Case labels must be **constant expressions** — no variables, no duplicates.
- Omitting `break` in switch causes **fall-through** to the next case.
- The ternary operator is an **expression**, so it can appear inside `printf()`.

12.3 Common Mistakes Table

#	Mistake	Correct Form
1	`if(n = 5)` (assigns, always true)	`if(n == 5)`
2	`if(condition);` (empty body)	`if(condition){ ... }`
3	`if(5 < x < 15)` (invalid range)	`if(x > 5 && x < 15)`
4	Missing `break` in switch (unintended fall-through)	Add `break;` after each case block
5	Float or string as switch expression	Use `if-else` ladder instead
6	Variable as a case label: `case x:`	Must be a constant: `case 5:`
7	Duplicate case values in same switch	Each case value must be unique

12.4 Frequently Asked Exam Questions

1. What is the difference between `if` and `if-else`?
2. What is a dangling-else problem?
3. What is fall-through in a switch statement? How is it prevented?
4. What is a case label? What are the rules for case labels?
5. Compare `switch` with `else-if` construct.
6. What is the ternary operator? How is it different from `if-else`?
7. What is short-circuit evaluation? Give an example.
8. Can a `switch` statement work with `float`? Explain.
9. What is the output of `switch` when no case matches and no `default` is present?
10. How are compound relational tests formed? Give an example.

12.5 One-Page Revision Summary

Decision Control in C — Complete Quick Reference

Construct	Syntax Fragment	Key Rule
`if`	`if(c) s;`	Executes s only when c is true
`if-else`	`if(c) s1; else s2;`	Exactly one of s1/s2 always runs
Nested if	`if(c1){ if(c2) s; }`	Inner tested only when outer is true
else-if construct	`if(c1)… else if(c2)… else…`	Tests in order; first match wins
Ternary	`c ? e1 : e2`	Expression — returns a value
switch	`switch(e){ case k: … }`	Equality match; fall-through without break

Logical: && AND, || OR, ! NOT | **Relational:** `< > <= >= == !=` | **In C:** 0=false, nonzero=true

Golden rules: Use `==` not `=` in conditions | Always add `break` in switch cases | Range checks need &&, not chained relational operators